LLM-based Mockless Unit Test Generation for Java

Qinghua Xu, Guancheng Wang, *Member, IEEE,*, Lionel Briand, *Fellow, IEEE*, Zhaoqiang Guo, Kui Liu, *Member, IEEE*

arXiv:2605.26851v1 [cs.SE] 26 May 2026

***Abstract*—Large language models (LLMs) have shown strong potential for automated test generation, yet most approaches to generating Java unit tests still rely on mocking frameworks to handle dependencies. Mockless test generation could exercise more real low-level code, but it faces challenges such as invalid test code generation due to hallucination, strict language constraints, and inadequate dependency awareness. We identify two root causes behind these hallucinations: *not knowing*, where the LLM lacks sufficient project-specific context, and *not following*, where the LLM fails to comply with constraints even when they are provided in the prompt.**

We present MocklessTester, a mockless unit test generation approach built around two strategies: *context-enriched generation* and *constraint-enforced fixing*. To mitigate *not knowing*, context-enriched generation mines real usage patterns from existing project code and leverages them to generate tests. To mitigate *not following*, constraint-enforced fixing performs two-stage repair under symbol-, protocol-, and iteration-level constraints, using a ClassIndex, a Markov typestate model, and experience memory.

We evaluate MocklessTester against the state-of-the-art baseline on Defects4J and Deps4J, a new post-cutoff benchmark containing more complex recent Java repositories. Experimental results show that MocklessTester improves line coverage by 19.99% and 22.69% and branch coverage by 24.90% and 15.78% on the two benchmarks, respectively, and improves mutation score by 13.67% and 0.17%. Beyond the class under test, MocklessTester also exercises more real dependency code, covering 378 and 55 additional lines in dependency classes, respectively. The improvement in test quality comes with higher total token and time costs than the baseline, since MocklessTester can continue making progress over more iterations rather than plateauing early. Nevertheless, the cost remains practical, averaging 108.97 seconds and 26.59k tokens per method on Defects4J, and 69.85 seconds and 25.46k tokens per method on Deps4J. Ablation results further confirm that all major components contribute positively to the final performance.

***Index Terms*—Large language models, automated test generation, mockless testing, Java, unit testing**

I. Introduction

UNIT testing plays a crucial role in modern software development because it helps software developers identify and fix defects early in the lifecycle [1, 2, 3]. However, manually creating tests is laborious and error-prone, motivating the development of automated test generation techniques [2, 4, 5, 6]. Traditional automated testing approaches (e.g., EvoSuite [4] and Randoop [5]) primarily rely on search-based algorithms to improve code coverage but struggle to produce semantically meaningful, human-readable tests [2, 7].

Qinghua Xu is with Lero, the Research Ireland Centre for Software, University of Limerick, V94 T9PX Limerick, Ireland (e-mail: qinghua.xu@ul.ie).

Guancheng Wang is with Lero, the Research Ireland Centre for Software, University of Limerick, V94 T9PX Limerick, Ireland (e-mail: guancheng.wang@ul.ie).

Lionel Briand is with the University of Ottawa, Ottawa, ON K1N 6N5, Canada, and also with Lero, the Research Ireland Centre for Software, University of Limerick, V94 T9PX Limerick, Ireland (e-mail: lbriand@uottawa.ca, lionel.briand@lero.ie).

Zhaoqiang Guo is with the State Key Laboratory of Blockchain and Data Security, Zhejiang University, Hangzhou, China (e-mail: gzq@smail.nju.edu.cn).

Kui Liu is with Software Engineering Application Technology Lab Huawei, Hangzhou, China (e-mail: brucekuiliu@gmail.com).

Recent advances in large language models (LLMs) have enabled a new generation of tools that generate syntactically valid and semantically meaningful test cases [7, 2, 6, 8]. However, much of the existing work focuses on weakly typed programming languages such as Python [9, 10, 11]. In contrast, generating tests for strongly typed languages such as Java remains significantly more challenging, due to the need for strict adherence to type constraints, complex language-specific syntax, and richer execution semantics [2].

Several approaches have been proposed for Java test generation [2, 6, 7], demonstrating substantial improvements in code coverage over traditional search-based approaches. However, a critical limitation pervades nearly all existing LLM-based test generation tools: many approaches operate at the method level, focusing on a single class under test (CUT) [2, 6], or rely heavily on *mocking frameworks* (e.g., Mockito, PowerMock) to isolate the method under test (MUT) from its dependencies [12]. In object-oriented programming, a **dependency** refers to any class, interface, module, or external component that a CUT relies on to perform its behavior. In this context, method-level test generation approaches are inherently limited to self-contained methods without external dependencies [2, 6]. For example, they are well-suited for testing pure functions such as a utility method that computes the factorial of an integer or performs string normalization, where all inputs are primitive types, and no external objects or variables are required. However, they struggle with methods that depend on external components, such as a service method that interacts with a database connection or invokes a parser, since these dependencies cannot be exercised without additional setup or simulation. To address this challenge, dependencies must either be instantiated or simulated. In both practice and existing research, mocking is widely adopted due to its efficiency and minimal requirement for understanding dependency implementations. Despite its practicality, mocking introduces two fundamental limitations:

- *Shallow coverage*. Mocked tests only exercise the MUT's logic in isolation. The actual behavior of dependency classes, i.e., the project classes that the CUT calls or instantiates, remains untested. For example, a service method that mocks a database call may pass tests even if the real query logic in the database layer is incorrect.
- *Mock fragility.* Mock-based tests are tightly coupled to the assumed API behavior. When dependencies evolve, mocked expectations break even if the real behavior is compatible [13]. For example, if a dependency method changes its internal implementation or response format while preserving its method signature, existing mocks may still fail due to outdated assumptions.

An alternative is **mockless testing**, where real dependencies are instantiated and executed rather than simulated. Unlike mock-based tests, which validate the CUT under simulated dependency behavior, mockless tests exercise concrete dependency implemen-

tations and the interactions between components. As a result, they can expose failures caused by invalid object construction, incorrect API call sequences, or mismatches between the CUT and its dependencies. However, this also makes test generation substantially more difficult. Key challenges include object instantiation, abstract type resolution, stateful objects compliance, and import resolution. Even with recent advancements in LLMs, solving these issues remains challenging primarily due to a combination of hallucination, strict language constraints, and inadequate dependency awareness. For example, LLMs may invent nonexistent factory methods, use incorrect constructor signatures, or call methods in invalid sequences, resulting in compilation errors or runtime exceptions and thereby severely limiting the effectiveness of mockless testing.

***Not knowing* vs. *Not following*.** We posit that these limitations are derived from two main sources: insufficient context and lack of constraint enforcement mechanisms. Without adequate context, LLMs cannot correctly infer dependency usage (e.g., valid constructors or factory methods), as they simply *do not know* the necessary project-specific information required to generate valid code. Meanwhile, even when the required constraints are provided, existing repair approaches rely solely on natural-language instructions and lack mechanisms to enforce that the generated fixes satisfy project-specific rules, such as valid symbols or legal API usage sequences. In this case, the model may know what to do but still *does not follow* the required constraints to produce a valid repair. This issue is particularly pronounced for smaller models (e.g., with fewer than 30B parameters), which are less able to reliably adhere to such constraints. Addressing these limitations requires complementary strategies: enriching generation with context to mitigate knowledge gaps and introducing explicit enforcement mechanisms to ensure compliance with constraints during repair.

In this work, we propose MocklessTester, a Java unit test generation approach that aims to exercise as much real project code as possible when testing a class. MocklessTester follows an iterative *plan–generate–validate–fix* workflow, inspired by recent advances in LLM-based test generation [2]. To enable effective mockless unit testing, we introduce two novel strategies that strengthen the generation and repair phases by addressing the limitations identified above.

- *Context-enriched Generation.* We incorporate class usage patterns retrieved by a program slicer into the test generation process. For every dependency of the class under test, the slicer mines real instantiation/usage patterns from the project and injects them into the generator's prompt, so that plausible-but-wrong object instantiations are less frequently generated in the first place.
- *Constraint-enforced Fixing*. We introduce a two-stage repair mechanism that first generates a fix from execution feedback and then re-generates under explicit symbol-, protocol-, and iteration-level constraints when violations are identified. In the second stage, the LLM must produce a detailed justification of how the revised fix satisfies all constraints and resolves the failure, enabling stronger constraint adherence during repair.

We evaluate MocklessTester on Defects4J, a widely used benchmark for Java test generation [7, 14] and the benchmark used by our baseline PANTA [7, 14], enabling a direct comparison with prior results. To avoid potential data leakage and to evaluate our approach on more realistic multi-module projects, we constructed a new dataset, Deps4J, by collecting recent Java projects released after the release of the LLM we employed (Qwen3-Coder). Unlike Defects4J, which consists of single-module projects, Deps4J contains more complex multi-module projects. Since mockless tests are intended to exercise real project dependencies, we also measure project dependency line coverage (DepLC), defined as the number of lines in dependency classes other than the CUT that are covered. Experimental results show that, on Defects4J, MocklessTester improves average line coverage from 68.83% to 88.82%, branch coverage from 58.84% to 83.74%, mutation score from 38.33% to 52.00%, and project DepLC from 819 to 1197; on Deps4J, it improves line coverage from 53.29% to 75.98%, branch coverage from 42.34% to 58.12%, and project DepLC from 224 to 279 while matching mutation score overall. Although the total cost in terms of tokens and time is higher, it remains practical, consuming 108.97 seconds and 26.59k tokens per method on Defects4J, and 69.85 seconds and 25.46k tokens per method on Deps4J. In addition, the ablation study shows that removing any major component degrades performance.

The contributions of this work are as follows:

- We identify *dependency usage patterns* as critical contextual information for mockless unit test generation, and propose a novel context-enriched prompting strategy that augments the LLM with real dependency instantiation and usage examples mined from the target project. This significantly reduces hallucinations caused by insufficient contextual knowledge.
- We propose a novel *constraint-enforced fixing* mechanism that corrects invalid tests at the symbol, protocol, and iteration levels through a two-stage repair process, in which the second stage requires the model to generate a structured justification explaining how the revised fix resolves the original failure and satisfies all constraints. This mechanism goes beyond test generation and directly targets "not following" hallucinations, a common and key bottleneck in many LLM-based tasks that require reliable adherence to explicit constraints.
- To the best of our knowledge, MocklessTester is the first LLM-based *mockless* unit test generation approach for Java. Unlike prior work that either relies on mocks or is limited to isolated method-level testing, MocklessTester enables test generation exercising project dependencies, advancing automated test generation from mocked unit testing toward realistic testing.
- We conduct a comprehensive empirical evaluation of both Defects4J and Deps4J using line coverage, branch coverage, mutation score, and a new metric, *DepLC*, which measures the amount of real project code exercised beyond the CUT and captures the extent to which dependencies are exercised. Results show that MocklessTester consistently outperforms existing state-of-the-art (SOTA) baselines across these metrics, demonstrating the effectiveness of mockless unit test generation.

II. Motivating Example

We present a motivating example adapted from Defects4J in Figure 1a and Figure 1b. Given the CUT `ToXmlGenerator`, a valid test must instantiate three external objects: an `IOContext`, a low-level

```java
// Excerpt of the class under test: ToXmlGenerator
public class ToXmlGenerator extends GeneratorBase {
    protected final XMLStreamWriter2 _xmlWriter;     // abstract type
    protected QName _nextName = null;                // must be set
                                                     // before any writeX(...)

    public ToXmlGenerator(IOContext ctxt, int stdFeatures, int xmlFeatures,
            ObjectCodec codec, XMLStreamWriter sw) {
        super(stdFeatures, codec);
        _ioContext = ctxt;
        _originalXmlWriter = sw;
        _xmlWriter = Stax2WriterAdapter.wrapIfNecessary(sw); // adapter
    }
    public void initGenerator() { /* writes XML prolog; must run first */ }
    public final void setNextName(QName name) { _nextName = name; }

    @Override public final void writeStartObject() throws IOException {
        _verifyValueWrite("start an object");
        _writeContext = _writeContext.createChildObjectContext();
        if (_nextName /*precondition*/ == null) _handleStartObject();
    }
    void _handleStartObject() throws IOException {
        if (_nextName == null) handleMissingName();      // throws!
        _xmlWriter.writeStartElement(_nextName.getNamespaceURI(),
                                     _nextName.getLocalPart());
    }
}
```

(a) Class under test: `ToXmlGenerator`.

```java
// Naive LLM-generated test - three latent failures
@Test public void testWriteStartObject() throws Exception {
    // (a) fabricated factory - no such method exists
    ToXmlGenerator gen = ToXmlGenerator.create();
    // (b) anonymous subclassing of abstract XMLStreamWriter2
    XMLStreamWriter2 w = new XMLStreamWriter2() { /* ...empty stubs... */ };
    // (c) writeStartObject() called without setNextName(...)
    gen.writeStartObject();                      // IllegalStateException
    assertNotNull(gen);
}
```

(b) A plausible but hallucinated LLM attempt

Fig. 1. Motivating example adapted from Defects4J.

`XMLStreamWriter` obtained from `XMLOutputFactory`, and an internal `Stax2WriterAdapter` wrapper. In addition, `setNextName(...)` must be invoked before any `writeStartX(...)` call to avoid a runtime `IllegalStateException`. While mocking frameworks can bypass these dependency instantiation and usage requirements by simulating external behaviors, they fail to exercise the real interactions among dependent components. In contrast, a mockless tester must generate executable tests that instantiate real dependencies and respect valid object usage constraints, making the process substantially more challenging and prone to invalid test generation. As shown in Figure 1b, when an LLM baseline is prompted with only the CUT source code, it generates plausible-looking tests exhibiting two types of generation errors mentioned in Section I.

Source I: Insufficient Context (*"Not Knowing"*). The LLM fabricates a `ToXmlGenerator.create()` factory and emits an empty anonymous subclass for `XMLStreamWriter2` because it has never seen how real Jackson-XML code constructs the generator or how the abstract writer field is populated. The actual pattern should be allocating an `XMLStreamWriter` via `XMLOutputFactory`, passing it to the five-argument constructor, where `Stax2WriterAdapter.wrapIfNecessary(...)` yields the concrete `XMLStreamWriter2`. This pattern is visible elsewhere in the source code but absent from the prompt. Such errors arise from *insufficient project-specific knowledge* and thus can be prevented by supplying that knowledge at the input. MocklessTester therefore address them at the generation phase, by mining the concrete usage patterns to instantiate an object.

Source II: Unenforced Constraints (*"Not Following"*). The call `gen.writeStartObject()` in Figure 1b is syntactically correct but fails at runtime because `_nextName` was never set before invocation. This reflects a broader failure mode in which the model does not reliably follow project-specific correctness constraints, even when the required information is available. Such failures manifest at three levels. At the *symbol level*, the model may invent non-existent classes or invoke methods with invalid signatures. At the *protocol level*, it may violate required API call sequences, such as omitting `setNextName(...)` before `writeStartObject()`. At the *iteration level*, the model may repeatedly generate ineffective fixes because it lacks a mechanism to retain and reuse successful repair patterns across repair attempts. To address this, MocklessTester performs a two-stage repair process. The initial fix is generated from the error signals, and then checked against symbol-, protocol-, and iteration-level constraints. If violations are identified, a second repair is triggered, requiring the model to provide a structured justification explaining why the revised fix satisfies these constraints and resolves the original failure.

III. Approach

MocklessTester consists of two stages: a one-time *preparation* stage that constructs project-specific artifacts, and an iterative multi-agent *plan–generate–validate–fix* loop that generates and repairs tests. The iterative loop of MocklessTester is designed around two core strategies that address the two sources of limitation identified in Section II.

In the preparation stage, MocklessTester analyzes the target repository to construct three project-specific artifacts that support different parts of the mockless pipeline. The code property graph (CPG) captures dependency usage in the repository and supports slicing-based retrieval of real object construction and API-call examples. The *ClassIndex* records project-visible classes, methods, constructors, and imports, providing a basis for detecting hallucinated or inconsistent symbols. The initial *Markov typestate model* captures likely valid method-call orders for stateful APIs, providing a basis for detecting illegal call sequences. Together, these artifacts connect context acquisition and constraint enforcement: the CPG helps the model understand how dependencies are used, while the ClassIndex and typestate model help ensure that generated or repaired tests adhere to project-specific symbol and protocol constraints.

After preparation, MocklessTester enters an iterative *plan–generate–validate–fix* loop composed of five agents: Initializer, Planner, Generator, Validator, and Fixer. The Initializer prepares the analysis artifacts and testing environment, the Planner creates a test plan by selecting uncovered CUT paths to target, the Generator produces candidate tests, the Validator compiles and executes generated tests, and the Fixer repairs failing tests based on validation feedback. Through this iterative collaboration, the framework progressively improves test quality and coverage over multiple rounds.

Two budgets control the iterative process: N_{fix} limits the number of repair attempts for each failed test, and N_{iter} limits the number of full *plan–generate–validate–fix* iterations. The process terminates when the target coverage is reached, when no improvement is observed in two consecutive iterations, or when N_{iter} is exhausted. The final output is the set of all passing tests accumulated during the iterative loop.

A. Stage I: Preparation

In this stage, MocklessTester constructs the project-specific artifacts required for the iterative test generation loop. Specifically, it builds a code property graph (CPG), a *ClassIndex*, and an initial

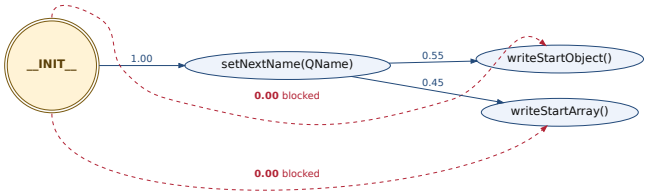


Fig. 2. Example of a typestate model for JacksonXML.

Markov typestate model. The CPG and ClassIndex are constructed once and reused throughout the workflow, while the typestate model is initialized in this stage and updated dynamically during subsequent iterations.

1) Joern CPG: Joern is a static analysis framework that represents source code as a queryable code property graph (CPG), integrating the abstract syntax tree, intraprocedural control flow, and call/data-flow relations [15]. We use Joern because MocklessTester requires concrete examples of how project classes are instantiated and invoked in real code, rather than only type signatures. Specifically, we run Joern on the target repository and use the resulting CPG as the substrate for program slicing, enabling backward slicing to reconstruct real usage patterns of the CUT and its dependencies.

2) ClassIndex: The *ClassIndex* records all classes visible under the project's build configuration, together with the members needed to validate generated tests. For each class reachable from the project source tree, test source tree, Maven-resolved dependency JARs, and the JDK, the index stores its fully-qualified name (FQN), simple name, package, accessible constructors, accessible methods, fields, and declared imports when source code is available. Thus, for each simple type name that may appear in a generated test, the ClassIndex maps the name to candidate FQNs drawn from three sources: (i) project classes, including `src/main/java` and `src/test/java`, (ii) dependency classes discovered from Maven-resolved JARs obtained with `mvn dependency:build-classpath`, and (iii) JDK classes from a static JDK table. The index filters candidates by classpath visibility and Java accessibility constraints, and ranks the remaining candidates based on package proximity to the CUT, explicit imports in the CUT, and source priority, with project-local symbols preferred over external-library symbols. During repair, the FIXER queries the ClassIndex to identify unresolved types, invalid constructor invocations, nonexistent method calls, and inconsistent imports, then proposes replacements from the ranked candidate set.

3) Typestate Model: For stateful classes whose public methods must be invoked in a particular order to avoid exceptions, we construct a Markov chain over method transitions. States represent the last method invoked on an object (i.e., the API's current usage context), with a distinguished `__INIT__` state representing the initial state before any method call. A directed edge $m \to m'$ means that method m' is a valid successor of method m, i.e., m' can be invoked immediately after m without violating the API usage constraints. For a current state m, the model assigns transition probabilities over all candidate successor methods m'' whose transitions from m have not been blocked. These probabilities are renormalized by the number of remaining valid successors after every update:

$$P(m \to m') \;=\; \frac{\mathbb{1}[(m,m') \notin \mathcal{B}]}{|\{m'' : (m,m'') \notin \mathcal{B}\}|}, \tag{1}$$

where $\mathcal{B}$ is the set of *blocked* transitions accumulated at runtime, and m'' ranges over candidate successor methods of m. A blocked transition is a method pair (m, m') that has been observed to violate the object's usage protocol, for example, because invoking m' immediately after m causes an `IllegalStateException` or another state-related failure. The numerator is 1 when the transition (m, m') remains valid, and 0 when it has been blocked; the denominator counts the candidate successors of m that remain valid. The chain is constructed *passively* from source-code evidence before test generation. We parse the CUT and its reachable dependency usages with tree-sitter, group method calls by receiver object, and add an edge between two methods when they appear consecutively on the same receiver in an observed usage sequence. For example, a sequence such as `writer.setNextName(q); writer.writeStartObject();` creates the transition `setNextName` $\to$ `writeStartObject`. We also inspect field-guarded preconditions (e.g. "`if _nextName==null throw IllegalStateException`") to infer required predecessor methods and to block transitions that would violate the guard. The chain is then updated *dynamically* from test outcomes (Section III-B5): every passing test reinforces the observed edges, and every state-related failure contributes a newly blocked edge to $\mathcal{B}$.

Figure 2 illustrates a typestate model derived from the motivating example in Section II. The solid edges from `setNextName(QName)` to the two `writeStartX()` methods represent valid method transitions, with probabilities assigned according to the out-degree normalization in Equation (1). In contrast, the dashed red edges from `__INIT__` directly to the `writeStartX()` methods represent blocked transitions in $\mathcal{B}$. These transitions are extracted statically from the precondition `if (_nextName == null) throw IllegalStateException`, which indicates that a name must be set before any start-write operation is invoked. Therefore, any candidate test that calls a `writeStartX()` method before `setNextName(QName)` is flagged by the FIXER as a protocol-level violation, as described in Section III-B5.

B. Stage II: Iterative Plan–Generate–Validate–Fix Loop

As depicted in Figure 3, this stage performs iterative test generation and repair through a multi-agent workflow, progressively producing, validating, and refining test cases until the termination criteria are met. We describe the agents in the order in which they are executed within each loop iteration.

1) INITIALIZER *(Optional):* The INITIALIZER runs once before the loop starts: it parses the CUT source with tree-sitter and emits a minimal test file containing the correct package declaration, the imports for the CUT and for JUnit, and an empty `@Test` placeholder per public CUT method. This "test skeleton" avoids wasting the first generation round on boilerplate and guarantees that every subsequent iteration edits a syntactically valid compilation unit. The step is optional. If an existing test file already exists (e.g. when MocklessTester is bootstrapped from a partial suite), MocklessTester appends new test cases instead of creating new test files.

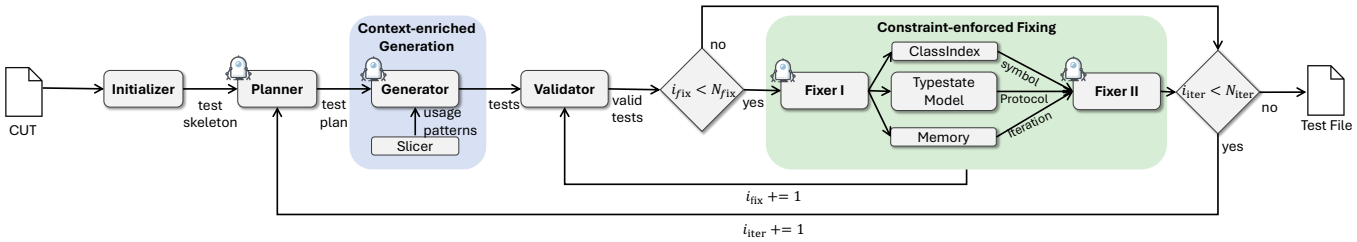


Fig. 3. Overview of MocklessTester.

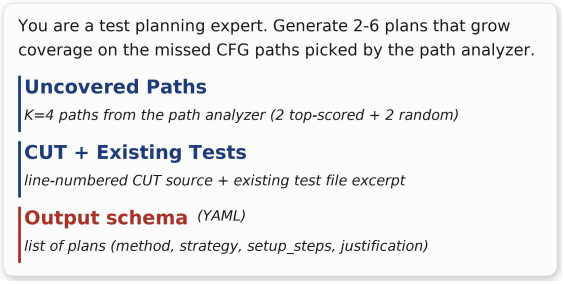


Fig. 4. Prompt Template for PLANNER

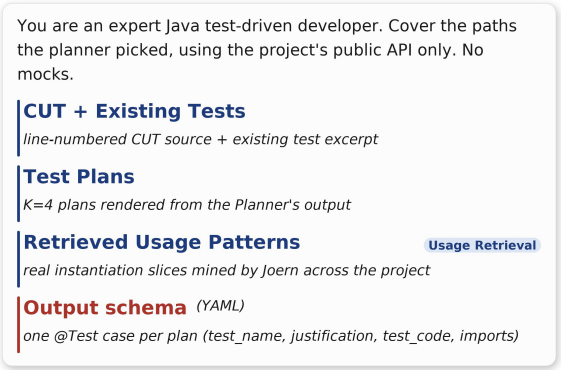


Fig. 5. Prompt template for GENERATOR

2) PLANNER*:* The PLANNER turns the current coverage gap into a concrete *test plan*. It builds a control-flow graph for every method of the CUT using COMEX [16], a tree-sitter-based code-view generation tool that extracts structural program representations such as control-flow and data-flow graphs, and then enumerates the resulting paths. Following the practice in PANTA [7] and to prevent overly long prompts, we restrict each iteration to a small path budget: we pick the top two paths from each of the two most-uncovered methods (exploitation) and two paths from two randomly sampled methods (exploration), yielding at most $K{=}4$ target paths per iteration.

Figure 4 shows the prompt template used by the PLANNER. The prompt provides the selected uncovered paths, the line-numbered class under test, and the current test file. Based on this information, the PLANNER generates 2–6 test plans, each targeting uncovered behavior in the selected paths. Each plan specifies the target method, the intended testing strategy, the required setup steps, and a short explanation of how the plan is expected to improve coverage.

3) GENERATOR*:* The GENERATOR is responsible for producing a candidate test. In a naive baseline, it only receives the CUT source code and the test plan, which often leads to hallucinated dependencies and incorrect instantiation patterns (the *"not knowing"* hallucination). MocklessTester addresses this by augmenting the Generator with a program slicer built on Joern, which enriches the prompt with real project usage patterns of both the CUT and its dependencies.

Given a dependency class D appearing in the CUT's signatures (constructor parameters, public-method parameters, fields, or return types), the slicer issues Joern queries to locate all call sites involving D across both the project's production and test code. For each call site, we perform backward slicing on the CPG to recover the minimal instantiation chain, including local variable definitions, factory invocations, and required imports. The resulting slices are deduplicated using structural hashing and ranked according to (i) occurrence in passing tests, (ii) occurrence in production code, and (iii) simplicity of the instantiation chain (favoring shorter sequences). The top-k slices are then rendered as executable Java code snippets and injected into the Generator prompt to guide correct dependency instantiation.

Figure 5 shows the prompt used by the GENERATOR. In addition to the class under test and the current test file, the prompt includes the test plans produced by the PLANNER and the retrieved usage patterns mined by the Joern-based slicer. These usage patterns provide concrete examples of how project classes are instantiated and used in existing code. The GENERATOR uses them to produce one `@Test` method for each plan, together with the required imports and a brief explanation. By supplying real dependency usage patterns, this prompt helps reduce errors caused by missing project-specific context ("not knowing"), such as hallucinated constructors, factory methods, or invalid object setup.

4) VALIDATOR*:* The VALIDATOR takes each candidate test produced by the Generator and determines whether it passes, fails, or should be forwarded to the Fixer. Concretely, it (i) compiles the test together with the existing test suite against the project's Maven-resolved classpath, (ii) executes it under JUnit with a per-test timeout, and (iii) analyzes the resulting compile-time and runtime diagnostics to produce a structured error report. This report is then forwarded to the Fixer. Tests that compile, execute, and pass are added to the accumulated test suite and used to update the typestate model via Equation (1).

5) FIXER*:* The FIXER addresses the second class of limitations (*"not following"*), which occur when the model fails to adhere to required constraints even when they are explicitly provided. Instead of relying on a single repair step, the Fixer performs a two-stage correction process for each failing test. The process

starts with the failing test and its compiled error signals from execution and compilation. MocklessTester first generates an initial repair based on this feedback (*Fixer I*). The resulting test is then checked against three constraint levels: (i) symbol-level constraints from the ClassIndex, which ground the repair in project-visible classes, constructors, methods, and imports before any higher-level reasoning is applied; (ii) protocol-level constraints from the Markov typestate model that enforce valid API call ordering; and (iii) iteration-level guidance from experience memory, which contains gold tests, fix recipes, and anti-patterns. Gold tests are previously generated tests that compile and pass, fix recipes summarize successful edits that repaired earlier failures, and anti-patterns record unfixable tests or repair attempts that should not be repeated. If any violation is detected, a second repair is triggered, conditioned on the identified constraint violations (FIXER II). In this second repair, the model must also produce a structured justification explaining why the revised test satisfies all constraints and resolves the original failure. This is a key design choice in our approach: by forcing the model to explicitly explain its own correction, we make constraint satisfaction part of the generation process rather than an implicit expectation. The justification requires the model to reflect on symbol-, protocol-, and iteration-level constraints, thereby strengthening adherence to them during repair.

Symbol-level constraints. Symbol-level failures, including fabricated classes, invalid method calls, incorrect constructor signatures, and illegal abstract instantiations, occur when the model produces identifiers that do not exist in the project. To eliminate such errors, the Fixer performs deterministic symbol verification using the ClassIndex.

For each failing test, all referenced symbols are resolved against the ClassIndex, and invalid references are replaced with valid candidates. Method calls are checked against the index entries, and mismatches are replaced with the closest valid alternative based on string similarity or removed if no safe replacement exists. Constructor invocations are similarly validated against the recorded constructor signatures, ensuring arity and type consistency.

Abstract or interface instantiations are replaced with concrete implementations retrieved from the ClassIndex based on type compatibility and package proximity. Missing or ambiguous imports are resolved using the same index, with project-local definitions prioritized over external dependencies.

Protocol-level constraints. Protocol-level failures arise when the generated test violates the required API usage order, such as invoking methods before required initialization steps. We address this by validating the test against the Markov typestate model.

Each method call is mapped to a state transition. Invalid transitions, defined as transitions with zero probability or as transitions along previously blocked edges, identify the first point of violation in the execution sequence. The FIXER then reconstructs a valid call ordering by replacing the offending segment with a feasible sequence that satisfies the typestate constraints. For example, consider a generated test that invokes `writeStartObject()` before calling `setNextName(QName)` on a streaming writer. According to the typestate model, the transition from the initial state directly to `writeStartObject()` is invalid because the required intermediate state induced by `setNextName` is missing. The Fixer identifies this violation and reconstructs the sequence by inserting the required initialization call, resulting in a valid ordering: `setNextName(QName)` followed by `writeStartObject()`.

Iteration-level constraints. The ClassIndex and typestate model define *what is valid*, but do not capture *which repair strategies are effective in practice*. Experience memory addresses this gap by retaining a structured record of past repair outcomes.

- *Successful repairs (gold memory):* Each successful fix is stored as a structured triple consisting of the error signature, the applied correction, and a before/after code diff. When a similar error recurs, the most similar prior case is retrieved and injected into the Fixer prompt as a concrete repair example.
- *Failed attempts:* Persistently failing patterns are stored as anti-patterns and fed back to the Generator as negative guidance, reducing repetition of known-invalid structures.
- *Unfixable cases:* Errors that exhaust N_{fix} repair attempts are recorded as unfixable patterns and optionally used to update the typestate constraints when applicable, preventing repeated exploration of infeasible behaviors.

Overall, the three components operate at complementary levels: the ClassIndex enforces correctness at the symbol level, the typestate model enforces correctness at the protocol level, and experience memory improves repair efficiency across iterations. Together, they shift constraint satisfaction from a purely language-driven process to a machine-enforced correction pipeline. Figure 6 shows the prompt template for FIXER I. This prompt provides the class under test, the current test file, and the diagnostics for failing tests, including compilation errors or Surefire execution failures. The goal of this stage is to repair simple failures directly from the observed error messages. The model returns a revised `@Test` method, any required imports, and a short explanation of the change. This first repair stage does not include symbol, typestate, or memory constraints; these checks are deferred to the second stage so that trivial errors can be fixed with a lightweight prompt.

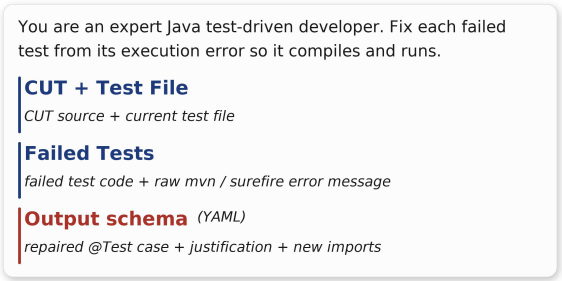


Fig. 6. Prompt Template for FIXER I

Figure 7 shows the second repair prompt, which is used when the first repair still violates project-specific constraints. In addition to the failed test and its diagnostics, this prompt may include three types of constraint information. The symbol check reports invalid or ambiguous classes, methods, constructors, or imports identified using the ClassIndex. The typestate check reports invalid call orders, required method sequences, and blocked transitions from the Markov typestate model. The experience memory provides relevant successful repairs and known anti-patterns from previous iterations. Using these inputs, the model generates a constraint-checked test and explains how the revised test addresses the reported violations. This makes the repair process explicitly

guided by project-specific constraints rather than relying only on the model's interpretation of error messages.

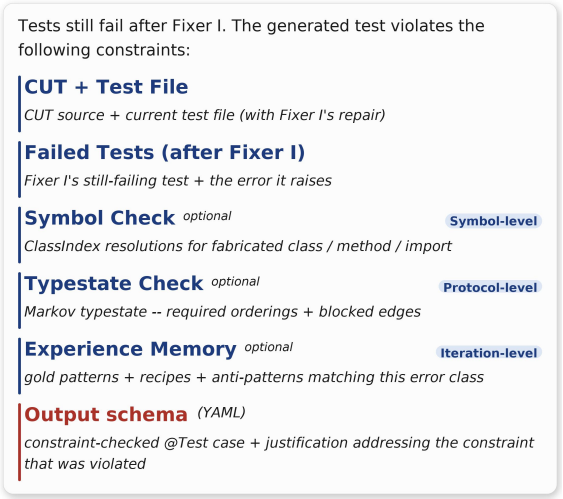


Fig. 7. Prompt Template for FIXER II

IV. EXPERIMENTAL DESIGN

A. Research Questions

We design our empirical evaluation around three research questions as follows.

- **RQ1 (Effectiveness):** How effective is MocklessTester in generating tests for Java repositories compared with the SOTA baseline?
- **RQ2 (Efficiency):** What efficiency cost does MocklessTester incur, in terms of wall-clock time and token consumption, relative to the baseline?
- **RQ3 (Ablation):** How much do the major strategies in MocklessTester, namely *usage retrieval*, *iteration-level memory*, *protocol constraints*, and *symbol constraints*, each contribute to its effectiveness?

For RQ1, we compare MocklessTester against the SOTA baseline, PANTA, to assess whether mockless test generation can improve the effectiveness of unit testing. This comparison examines both the quality of valid tests produced and the extent to which the generated tests exercise production code, including the CUT and code reached through real dependency interactions. The detailed effectiveness metrics are introduced in Section IV-D.

For RQ2, we analyze the practical cost of applying MocklessTester. Since mockless test generation requires LLMs to reason about real dependencies, construct valid object states, and iteratively repair generated tests, it may introduce additional overhead compared to PANTA. We therefore evaluate the computational cost of test generation in terms of time and token consumption, with detailed efficiency metrics presented in Section IV-D.

For RQ3, we conduct an ablation study to quantify how the major components of MocklessTester support its two core strategies: context-enriched generation and constraint-enforced fixing. We compare the full approach with four variants. The first variant, *No_UR*, removes usage retrieval and therefore directly ablates the context-enriched generation strategy by providing no mined dependency-usage examples to the generator. The other three variants ablate the constraint-enforced fixing strategy at different levels: *No_Sym* removes symbol-level constraints, so the repair stage is no longer explicitly restricted to valid project-visible classes, methods, and fields; *No_Prot* removes protocol-level constraints, allowing the model to violate valid API call sequences and object-state transitions; and *No_Iter* disables iteration-level constraints, i.e., experience memory, so information from gold tests, successful fix recipes, and unfixable anti-patterns is no longer carried into later repair iterations. These variants are chosen because they isolate the retrieval component used to address *not knowing* and the three complementary constraint levels used to address *not following*.

B. Datasets

We evaluate MocklessTester on two complementary benchmarks: Defects4J, a widely used Java test-generation benchmark that enables direct comparison with prior work, and Deps4J, a new, more challenging benchmark curated from post-cutoff open-source projects. While Defects4J provides a standard basis for evaluation, it may suffer from potential data leakage and relies on older Java and JUnit versions. To address these limitations, we construct Deps4J to (i) reduce the risk of data leakage, (ii) evaluate whether MocklessTester generalizes across diverse projects and dependency structures, and (iii) assess its effectiveness under more modern Java environments and more complex multi-module repository scenarios. Table I summarizes the main characteristics of the two benchmarks, including the number of projects ("#Projects"), selected classes ("#Classes") and methods ("#MUTs"), average cyclomatic complexity ("CC"), Java and JUnit versions, and the earliest project creation time ("Created After").

TABLE I
CHARACTERISTICS OF DEFECTS4J AND DEPS4J.

Benchmark	#Projects	#Classes	#MUTs	CC	Java	JUnit	Created After
Defects4J	14	130	2971	16.5	11	4	2008
Deps4J	5	30	540	38.4	17	5	2025

Defects4J. Following recent research [7, 14], we use the Defects4J v2.0.0 bug-free revisions of 14 Java projects. We focus on the 130 non-trivial classes identified by Gu et al. [7] as having cyclomatic complexity (CC) ≥ 11 (i.e., we exclude simple classes that admit trivial single-path tests). These projects span Apache Commons (`Lang`, `Math`, `Csv`, `Cli`, `Codec`, `Collections`, `Compress`), Jackson (`JacksonCore`, `JacksonDatabind`, `JacksonXml`), `Jsoup`, `JxPath`, `Gson`, and `JodaTime`. All projects target Java 11 and JUnit 4. Their public commit dates range from 2008 to 2018, preceding the release date of any LLM considered in this study. We therefore cannot rule out potential data leakage in this dataset.

TABLE II
STATISTICS OF THE DEPS4J DATASET.

Project	Domain	#CUTs	#MUTs	CC
adk-java [17]	Agent Development Kit (Google)	6	118	38.7
fit-framework [18]	Model-engineering utils (Huawei)	6	89	17.8
agentscope-java [19]	Multi-agent runtime	6	174	87.0
a2a-java [20]	Agent-to-Agent SDK (Google)	8	107	29.0
jquick-curl [21]	ANTLR-generated curl parser	4	52	14.8
Total		**30**	**540**	**38.4**

Deps4J. To mitigate the data leakage threat inherent in Defects4J, and to evaluate MocklessTester on software that reflects modern Java practice, we curate Deps4J: a new benchmark of 30 classes drawn from 5 high-profile open-source projects whose first commit dates and complete development histories post-date the training-data cutoff of the LLM used in our experiments (Qwen3-Coder, cutoff 2025-03). The selected projects (see Table II) include `adk-java` (Google Agent Development Kit), `fit-framework` (Huawei Model Engine), `agentscope-java`, `a2a-java` (Google Agent-to-Agent SDK), and `jquick-curl`. We selected these projects because they (i) have been public after the year 2025, (ii) target Java 17 with JUnit 5, exercising new language features that were unavailable in the Defects4J projects, and (iii) are actively starred and forked on GitHub, indicating engineering quality comparable to Defects4J subjects.

C. Baseline

We compare MocklessTester with PANTA, the recent SOTA test generation tool for Java programs. Prior work spans search-based approaches (e.g., EvoSuite [4]), random-based approaches (Randoop [5]), and LLM-based approaches [22, 23, 7, 2]. While EvoSuite and Randoop have been widely used as baselines, recent studies have shown that LLM-based approaches produce more semantically meaningful and higher-quality test cases [22, 23, 7, 2, 14, 24]. Among LLM-based approaches, CANDOR [2] and PANTA [7] are the most relevant and strongest approaches. CANDOR proposes the first multi-agent LLM-based test-generation framework for Java programs, demonstrating the effectiveness of decomposing the task into specialized agents, but it is limited to method-level programs and does not handle external dependencies. In contrast, PANTA significantly advances Java test generation beyond method-level and establishes the current SOTA results on a widely adopted benchmark dataset Defects4J. However, PANTA still does not explicitly handle external dependencies in complex Java repositories.

D. Evaluation Metrics & Statistical Testing

We evaluate generated test suites from three perspectives: *effectiveness*, *efficiency*, and *statistical significance*. Effectiveness metrics measure how well the generated tests exercise the production code and expose behavioral differences. Efficiency metrics quantify the computational cost of producing the tests, including time, token consumption, and effort for iterative refinement. Finally, we apply statistical testing to account for the randomness in LLM-based generation and to assess whether the observed differences between approaches are statistically significant.

1) Effectiveness metrics: We adopt the following metrics to evaluate the effectiveness:

Number of Tests. This metric reports the number of valid test cases in the final generated test file. We include this metric to quantify the number of executable tests produced by each approach. In unit test generation, generated tests may be discarded because they fail to compile, fail during execution, violate framework constraints, or rely on invalid dependency interactions. Thus, a larger number of valid tests indicates that the approach produces more runnable tests during generation and repair.

Line and Branch Coverage. We record standard JaCoCo line coverage and branch coverage on the CUT. These two metrics quantify how thoroughly the suite exercises the CUT in isolation and are directly comparable to the numbers reported by prior LLM-based generators [7, 2].

Mutation Score. Line and branch coverage measure *reach* but not *sensitivity*: a test that invokes a method without asserting anything meaningful will still increase coverage. We therefore also report the mutation score computed with PIT [25] using the default operator set. For a CUT with N generated mutants, the score is the fraction that are killed by at least one test in the generated suite: $MS = |\text{killed}|/N$.

Dependency Line Coverage (DepLC). Line and branch coverage are computed *only on the CUT*. A mockless suite, however, also exercises the CUT's real dependencies. The central motivation of this work is that these dependencies constitute the "shallow-coverage" gap left by mocking-based approaches (Section I). To quantify this effect, we extend JaCoCo's class-level reporting to the entire Maven module containing the CUT, and define three derived metrics over the resulting coverage vector:

$$DLC = \sum_{\ell \in CUT} \mathbb{1}[\ell \text{ covered}], \tag{2}$$

$$TLC = \sum_{\ell \in Module} \mathbb{1}[\ell \text{ covered}], \tag{3}$$

$$DepLC = TLC - DLC, \tag{4}$$

where DLC (Direct Line Coverage) counts the covered lines in the CUT itself, TLC (Transitive Line Coverage) counts the covered lines across all production classes in the CUT's Maven module, and $DepLC$ isolates the additional covered lines outside the CUT. Thus, $DepLC$ captures the extent to which generated tests execute real project code beyond the target class. This complements the CUT-level line coverage, branch coverage, and mutation score: while those metrics evaluate how thoroughly the target class is tested, $DepLC$ measures the additional dependency code reached through mockless execution and, indirectly, the extent to which dependencies are exercised.

2) Efficiency metrics: We adopt the following metrics to evaluate the efficiency:

Number of Iterations. This metric reports the number of `plan-generate-validate-fix` cycles executed for a CUT before the tool stops, either by reaching its iteration budget, satisfying the stopping criterion, or failing to improve further. We include this metric to measure the convergence efficiency of each approach.

Tokens Per Method. This metric reports the average token consumption per method in the CUT. We include this metric to measure the cost efficiency of each approach at the granularity of the testing method. Token consumption directly reflects the computational and monetary cost of producing tests. Normalizing token usage by the number of methods in the CUT makes the comparison less sensitive to differences in class or project size, and provides a clearer view of how much LLM budget is required to test each unit of code.

Tokens Per Iteration. This metric is calculated as the total token consumption divided by the number of iterations. We include this metric to measure the cost efficiency of each approach at the iteration level. In iterative test generation, achieving higher coverage often requires additional iterations, thereby increasing the total number of tokens consumed. Therefore, a larger total token count may reflect more extensive refinement rather than lower efficiency. By normalizing token consumption per iteration,

this metric captures the average token cost per generation cycle and provides a fairer comparison of per-iteration efficiency across approaches.

Time Per Method. This metric is calculated as the total wall-clock time divided by the number of methods in the CUT, where the total time is the elapsed time from the start to the end of test generation for the CUT. We include this metric to measure the time required to test each unit of code. Since different CUTs may contain different numbers of methods, the total time alone can be strongly affected by target size and may not provide a fair comparison across CUTs or projects. By normalizing wall-clock time per method, this metric provides a clearer view of the average generation cost per method and reflects the practical scalability of an approach when applied to larger code units.

Time Per Iteration. This metric is calculated as the total wall-clock time divided by the number of generate–validate–fix iterations, where the total time records the elapsed time from the beginning to the end of test generation for a CUT. We include this metric because total time alone does not distinguish between approaches that perform different numbers of cycles. In iterative test generation, additional iterations may be necessary to improve the test suite and reach higher coverage, which naturally increases total runtime. Therefore, a larger total time may reflect more extensive refinement rather than lower efficiency. By normalizing time to the number of iterations, this metric captures the average time cost per cycle and enables a fairer comparison of per-iteration efficiency across approaches.

3) Statistical Testing: LLMs are inherently stochastic, as different runs may produce different tests even under identical experimental settings. To mitigate the influence of randomness, we report the average results on the Defects4J (130 CUTs) and Deps4J (30 CUTs) datasets. We also conduct pairwise statistical testing and calculate effect sizes for each metric.

Following the guidance for statistical testing [26], we use the Wilcoxon signed-rank test to compare paired results between MocklessTester and the baseline PANTA. In addition to significance testing, we report the Vargha–Delaney $\hat{A}_{12}$ effect size [27], which is widely used for randomized software-engineering algorithms [26]. In our context, $\hat{A}_{12}$ estimates the probability that MocklessTester outperforms PANTA on a randomly selected paired observation, with ties counted as half. A value of 0.5 indicates no preference between the two approaches; values above 0.5 indicate that MocklessTester tends to achieve higher values, while values below 0.5 indicate that PANTA tends to achieve higher values. Following Vargha and Delaney [27], we interpret the effect as *negligible* when $|\hat{A}_{12} - 0.5| < 0.06$, *small* when $0.06 \leq |\hat{A}_{12} - 0.5| < 0.14$, *medium* when $0.14 \leq |\hat{A}_{12} - 0.5| < 0.21$, and *large* when $|\hat{A}_{12} - 0.5| \geq 0.21$.

E. Implementation Details

All experiments are conducted on a Precision 7960 Tower workstation equipped with an Intel Xeon w9-3495X processor and dual NVIDIA RTX 6000 Ada GPUs. The implementation is written in Python and uses LangChain [28] for LLM integration. We use Qwen3-Coder-30B-A3B-Instruct-FP8 [29] as the backbone model and serve it locally with vLLM [30]. This local deployment avoids API costs and privacy concerns associated with closed-source LLM services, making the experimental setting more suitable for testing on real projects. To facilitate replication, we plan to release our code and data upon acceptance.

To ensure a fair comparison, both MocklessTester and PANTA are configured under identical settings. Specifically, both tools use a context window of $16\,384$ tokens, a sampling temperature of 0.2, and a maximum output length of 4096 tokens, as in PANTA [7]. We set the maximum number of generation iterations N_{iter} to 30, the maximum number of repair N_{fix} attempts to 5, and the plateau-based early-stopping patience to 4. These values were selected based on the PANTA experimental results, where test generation for most CUTs converged under this configuration. Using the same generous budget allows us to fully exercise the capability of both PANTA and MocklessTester and ensures that the comparison is not biased by premature stopping. For both tools, the generation target is set to 100% line coverage. Test generation stops once this target is reached or when the iteration budget or plateau criterion is triggered. For the program slicer used in MocklessTester, we adopt Joern, a widely used code analysis framework that supports code property graph-based analysis. For mutation testing, we use the default set of mutation operators, following the practice of PANTA [7].

V. Experimental Results

A. RQ1 Results (Effectiveness)

Table III reports the experimental results of RQ1, where we compare MocklessTester with PANTA in the effectiveness of generating test cases. Overall, MocklessTester outperforms PANTA on both Defects4J and Deps4J in line coverage, branch coverage, mutation score, and project DepLC. The Wilcoxon signed-rank tests indicate that all pooled improvements are statistically significant ($p < 0.05$), with large effect sizes ($|\hat{A}_{12} - 0.5| \geq 0.21$).

On Defects4J, MocklessTester generates 297 fewer tests per project on average than PANTA (402.79 vs. 699.57), while achieving higher line and branch coverage on every project. In particular, MocklessTester improves average line coverage by $19.99\,pp$ and average branch coverage by $24.90\,pp$. Even its lowest project-level coverage remains high, with $80.23\,\%$ line coverage on JXml and $74.69\,\%$ branch coverage on JCore, suggesting that MocklessTester is practically effective for real-world mockless Java unit testing. MocklessTester also achieves higher mutation scores on 13 of the 14 Defects4J projects, with Codec as the only exception ($\Delta = -3.14\,pp$). On average, it improves mutation score by $13.67\,pp$, indicating that the generated tests are not only broad in coverage but also strong in fault-detection capability. Finally, MocklessTester increases project DepLC by 378 lines on average (1197 vs. 819), which is consistent with its mockless design: by executing real dependencies rather than mocked surrogates, it exercises substantially more non-CUT code.

We observe similar patterns on Deps4J. MocklessTester again generates fewer tests per project on average than PANTA (114.4 vs. 190.8), while improving average line coverage by $22.69\,pp$ and average branch coverage by $15.78\,pp$. The overall mutation scores are nearly identical ($47.09\,\%$ for MocklessTester vs. $46.92\,\%$ for PANTA). At the project level, MocklessTester performs better on AgentScope, FIT, and JQuick, while PANTA performs better on A2A and ADK. One reason is that PANTA generates substantially more tests on Deps4J, giving it more opportunities to introduce assertions and kill mutants. This gives PANTA an inherent advantage in mutation score, which is sensitive not only to exercised code but also to the number and strength of test oracles. In addition, Deps4J contains more complex projects, many of

TABLE III
COMPARISON OF MOCKLESSTESTER AND PANTA ON TEST-SUITE SIZE AND EFFECTIVENESS METRICS. EACH METRIC SPANS THREE COLUMNS: MOCKLESSTESTER (M), PANTA (P), AND $\Delta = M - P$. MUTATION SCORE COLUMNS SHOW RAW $killed/total$ COUNTS IN PARENTHESES. THE BOTTOM *Stat. test* ROW REPORTS THE TWO-SIDED PAIRED WILCOXON SIGNED-RANK *p-value* COMPARING M AGAINST P ACROSS ALL PROJECTS AND THE VARGHA–DELANEY $\hat{A}_{12}$ EFFECT SIZE. "–" DENOTES NOT APPLICABLE.

Dataset	Project	Number of Tests			Line Coverage			Branch Coverage			Mutation Score			Project DepLC		
		M	P	Δ	M	P	Δ	M	P	Δ	M	P	Δ	M	P	Δ
Defects4J	Cli	65	75	-10	**96.18**	78.13	+18.05	**89.88**	57.44	+32.44	**57.23 (90/173)**	43.93 (72/173)	+13.30	**138**	131	+7
	Codec	196	284	-88	**90.95**	83.92	+7.03	**86.57**	74.29	+12.28	37.37 (449/1207)	**40.51 (486/1207)**	-3.14	415	**420**	-5
	Collections	387	351	+36	**93.41**	70.79	+22.62	**90.60**	67.75	+22.85	**72.86 (515/711)**	57.81 (410/711)	+15.05	717	**781**	-64
	Compress	197	524	-327	**85.89**	55.77	+30.12	**78.77**	49.43	+29.34	**40.76 (601/1570)**	24.01 (369/1570)	+16.75	**2368**	17	+2351
	Csv	142	254	-112	**96.52**	69.40	+27.12	**87.90**	55.25	+32.65	**29.36 (120/419)**	21.72 (85/419)	+7.64	**84**	69	+15
	Gson	162	619	-457	**86.86**	74.98	+11.88	**81.96**	60.81	+21.15	**56.55 (430/771)**	53.96 (410/771)	+2.59	587	**776**	-189
	JCore	394	486	-92	**80.49**	51.02	+29.47	**74.69**	44.79	+29.90	**34.42 (1148/3364)**	14.57 (485/3364)	+19.85	**1311**	620	+691
	JDatabind	417	770	-353	**81.64**	55.29	+26.35	**76.10**	46.74	+29.36	**60.22 (869/1443)**	41.72 (602/1443)	+18.50	**3133**	1833	+1300
	JXml	219	97	+122	**80.23**	64.83	+15.40	**80.12**	54.14	+25.98	**47.09 (306/652)**	23.47 (151/652)	+23.62	**221**	21	+200
	Jsoup	254	593	-339	**89.76**	78.30	+11.46	**80.42**	63.35	+17.07	**58.84 (566/979)**	37.18 (362/979)	+21.66	**1256**	1150	+106
	JxPath	426	584	-158	**84.45**	52.29	+32.16	**80.27**	45.33	+34.94	**58.48 (783/1351)**	50.85 (675/1351)	+7.63	**589**	349	+240
	Lang	1153	2281	-1128	**86.39**	71.12	+15.27	**83.00**	64.22	+18.78	**49.06 (2027/4209)**	27.89 (1141/4209)	+21.17	**352**	**352**	+0
	Math	1075	2244	-1169	**92.72**	74.09	+18.63	**89.10**	65.50	+23.60	**48.95 (3937/8155)**	35.27 (2837/8155)	+13.68	**3410**	2902	+508
	Time	552	632	-80	**97.94**	83.63	+14.31	**92.94**	74.67	+18.27	**76.85 (1032/1348)**	63.72 (857/1348)	+13.13	**2172**	2047	+125
	Average	402.79	699.57	-297	**88.82**	68.83	+19.99	**83.74**	58.84	+24.90	**52.00**	38.33	+13.67	**1197**	819	+378
Deps4J	A2A	92	225	-133	**87.04**	69.31	+17.73	**59.35**	55.39	+3.96	43.46 (93/214)	**59.35 (127/214)**	-15.89	**58**	1	+57
	ADK	123	257	-134	76.25	**78.25**	-2.00	**61.31**	60.96	+0.35	48.32 (144/298)	**55.03 (164/298)**	-6.71	101	**145**	-44
	AgentScope	173	226	-53	**76.52**	44.76	+31.76	**69.02**	33.83	+35.19	**54.45 (300/551)**	43.19 (238/551)	+11.26	**717**	534	+183
	FIT	112	237	-125	**79.76**	66.06	+13.70	**70.62**	57.90	+12.72	**79.19 (235/298)**	74.83 (222/298)	+4.36	**490**	411	+79
	JQuick	72	9	+63	**60.34**	8.07	+52.27	**30.29**	3.62	+26.67	**10.05 (55/547)**	2.19 (7/547)	+7.86	**27**	**27**	+0
	Average	114.4	190.8	-76	**75.98**	53.29	+22.69	**58.12**	42.34	+15.78	**47.09**	46.92	+0.17	**279**	224	+55
Stat. test	*p-value*	–	–	<.001	–	–	<.001	–	–	<.001	–	–	0.001	–	–	0.010
	$\hat{A}_{12}$	–	–	0.158	–	–	0.947	–	–	1.000	–	–	0.842	–	–	0.737

which are multi-module and involve richer dependency interactions. While mockless testing helps MocklessTester execute more non-CUT code, producing precise assertions for such complex behaviors remains challenging. Thus, achieving a comparable mutation score with fewer tests suggests that MocklessTester provides competitive fault-detection capability while substantially improving line, branch, and dependency coverage. Finally, MocklessTester improves project DepLC by an average of 55 lines (279 vs. 224). The consistent improvements on both Defects4J and Deps4J suggest that MocklessTester is not overfit to a specific benchmark, but generalizes across Java repositories from different domains. Moreover, because the Deps4J projects were created after the training cutoff date of Qwen3-Coder, the improvements are unlikely to be due to data leakage; rather, they are attributable to the design of MocklessTester.

Particularly, both MocklessTester and PANTA achieve their worst overall performance on JQuick. For MocklessTester, line coverage and branch coverage on JQuick are only $60.34\,\%$ and $30.29\,\%$, respectively, substantially lower than its Deps4J averages of $75.98\,\%$ and $58.12\,\%$. PANTA shows an even sharper drop, with line and branch coverage decreasing to $8.07\,\%$ and $3.62\,\%$, respectively. The mutation score and DepLC also reach their lowest values among all 19 projects. This result is surprising because JQuick has the lowest cyclomatic complexity among the five Deps4J projects, as shown in Table II. JQuick is a Java HTTP client framework that translates cURL commands into executable HTTP requests, allowing developers to issue requests without manually constructing low-level HTTP client code. Effective testing of such parser-centric code requires inputs that satisfy the underlying cURL grammar. However, this grammar is not available in JQuick's source code or its dependencies. As a result, both PANTA and MocklessTester often generate malformed or uninformative cURL strings. Moreover, parser feedback is usually coarse-grained, such as generic parse failures, and provides little actionable guidance about which grammar rule was violated or how the input should be revised. Consequently, the *plan–generate–validate–fix* loop becomes much less effective than it is for other projects, where failures often expose clear exceptions, return values, or state changes. This explains why both tools perform poorly on JQuick despite its relatively low cyclomatic complexity.

This finding suggests an important direction for future work: combining MocklessTester with grammar-aware or fuzzing-assisted input generation. For parser-centric components, fuzzers can systematically explore valid and near-valid inputs, while MocklessTester can use the resulting executions, failures, and seed inputs to construct valid test cases and assertions. Such a hybrid design could make the feedback loop more informative for grammar-driven code.

Answer to RQ1: MocklessTester is effective in generating tests for Java repositories, establishing new SOTA results in line coverage, branch coverage, mutation score, and DepLC. Statistical testing shows that all improvements are significant with large effect sizes.

B. RQ2 Results (Efficiency)

Table IV compares the efficiency of MocklessTester and PANTA. Overall, MocklessTester consumes more total LLM tokens and wall-clock time than PANTA. This overhead is expected because MocklessTester continues generating for substantially more iterations than PANTA, which tends to stop earlier. The Wilcoxon signed-rank tests show that the differences between MocklessTester and PANTA are statistically significant for all efficiency metrics (*p-value*≤ 0.05), with large effect sizes ($|\hat{A}_{12} - 0.5| > 0.21$), except for *Tokens/Iteration*, where the effect size is medium ($\hat{A}_{12} = 0.684$).

On Defects4J, MocklessTester consumes 26.59k tokens per method on average and takes 108.97 seconds per method. In comparison, PANTA consumes 5.74k tokens per method and takes

TABLE IV
COMPARISON OF MOCKLESSTESTER AND PANTA ON COST / EFFICIENCY METRICS. EACH METRIC SPANS THREE COLUMNS: MOCKLESSTESTER (M), PANTA (P), AND $\Delta = M - P$. TOKEN-BASED COLUMNS ARE REPORTED IN THOUSANDS. TOKENS-PER-METHOD AND TIME-PER-METHOD NORMALISE COST BY OUTPUT VOLUME; THE PER-ITERATION COLUMNS NORMALISE BY GENERATION EFFORT. THE BOTTOM *Stat. test* ROW REPORTS THE TWO-SIDED PAIRED WILCOXON SIGNED-RANK *p-value* COMPARING M AGAINST P ACROSS ALL PROJECTS AND THE VARGHA–DELANEY $\hat{A}_{12}$ EFFECT SIZE: $\hat{A}_{12} > 0.5$ INDICATES M USES MORE, < 0.5 INDICATES P USES MORE. "–" DENOTES NOT APPLICABLE.

Dataset	Project	Mean Iterations			Tokens / Method (k)			Tokens / Iteration (k)			Time / Method (s)			Time / Iteration (s)		
		M	P	Δ	M	P	Δ	M	P	Δ	M	P	Δ	M	P	Δ
Defects4J	Cli	15	2.5	+12.50	27.90	2.81	+25.10	60.45	42.09	+18.37	109.25	192.02	-82.77	236.71	2880.33	-2643.62
	Codec	14	9.86	+4.14	19.10	10.42	+8.68	38.20	42.88	-4.68	95.31	55.21	+40.10	190.61	227.25	-36.64
	Collections	19.2	4	+15.20	10.81	2.10	+8.71	43.57	36.90	+6.67	69.85	35.32	+34.53	281.58	619.78	-338.20
	Compress	15.67	9	+6.67	51.46	11.74	+39.72	71.90	75.95	-4.06	221.63	45.12	+176.51	309.65	291.90	+17.75
	Csv	20	8	+12.00	30.95	4.84	+26.11	73.25	51.24	+22.01	116.59	15.52	+101.07	275.93	164.25	+111.68
	Gson	15.75	10.5	+5.25	28.56	5.10	+23.46	73.44	75.13	-1.69	108.39	12.43	+95.96	278.71	183.21	+95.50
	JCore	18.67	6.33	+12.34	26.75	5.78	+20.96	62.72	49.31	+13.41	101.32	51.54	+49.78	237.63	439.41	-201.78
	JDatabind	19.33	7.56	+11.77	30.72	5.39	+25.34	73.63	61.01	+12.62	91.58	32.01	+59.57	219.48	362.44	-142.96
	JXml	22.75	3.75	+19.00	31.70	5.32	+26.38	76.28	34.37	+41.91	101.50	209.17	-107.67	244.27	1352.65	-1108.38
	Jsoup	15.5	4.62	+10.88	26.36	2.38	+23.98	54.00	38.14	+15.85	124.18	48.41	+75.77	254.37	775.94	-521.57
	JxPath	17.08	9.92	+7.16	34.85	11.59	+23.26	72.43	56.88	+15.55	123.38	57.11	+66.27	256.39	280.28	-23.89
	Lang	21.29	8.94	+12.35	15.92	3.83	+12.09	50.71	57.50	-6.79	68.80	12.77	+56.03	219.12	191.65	+27.47
	Math	13.7	6.4	+7.30	20.67	3.16	+17.50	54.05	36.97	+17.08	120.74	26.73	+94.01	315.81	312.42	+3.39
	Time	15	7.64	+7.36	16.50	5.85	+10.65	55.20	44.03	+11.16	72.99	19.34	+53.65	244.18	145.54	+98.64
	Average	17.35	7.07	+10.28	26.59	5.74	+20.85	61.42	50.17	+11.25	108.97	58.05	+50.92	254.60	587.65	-333.05
Deps4J	A2A	6.75	6.25	+0.50	28.34	12.86	+15.48	48.29	57.89	-9.61	81.06	67.16	+13.90	138.10	302.22	-164.12
	ADK	9.83	9.67	+0.16	31.48	17.36	+14.12	65.62	76.91	-11.29	86.37	67.41	+18.96	180.06	298.71	-118.65
	AgentScope	15.33	6.33	+9.00	38.00	9.70	+28.30	71.46	57.71	+13.75	90.90	38.33	+52.57	170.92	227.95	-57.03
	FIT	5.83	6.4	-0.57	17.04	6.49	+10.55	54.53	48.08	+6.45	50.34	28.17	+22.17	161.10	208.60	-47.50
	JQuick	8.5	2	+6.50	12.45	2.25	+10.19	26.36	10.13	+16.23	40.58	16.76	+23.82	85.93	75.44	+10.49
	Average	9.25	6.13	+3.12	25.46	9.73	+15.73	53.25	50.14	+3.11	69.85	43.57	+26.28	147.22	222.58	-75.36
Stat. test	*p-value*	–	–	$<.001$	–	–	$<.001$	–	–	0.005	–	–	0.009	–	–	0.029
	$\hat{A}_{12}$	–	–	0.947	–	–	1.000	–	–	0.684	–	–	0.895	–	–	0.368

58.05 seconds. Although both tools are given the same iteration budget, i.e., 30 iterations with plateau-based early stopping patience of 4, PANTA stops much earlier on average. This indicates that the higher total cost of MocklessTester is mainly due to its longer generation process, which allows it to continue improving coverage after PANTA has plateaued. When normalizing by the number of iterations, the token gap becomes much smaller: MocklessTester consumes only 11.25k more tokens per iteration than PANTA. More importantly, MocklessTester is substantially faster per iteration, requiring 254.60 seconds per iteration compared with 587.65 seconds for PANTA, a reduction of approximately $58.7\,\%$. This suggests that although MocklessTester performs more iterations, each iteration is more time-efficient.

The results on Deps4J show a similar trend. MocklessTester consumes more tokens and time per method than PANTA, with 25.46k vs. 9.73k tokens and 69.85 vs. 43.57 seconds, respectively. However, after accounting for the number of iterations, the token gap becomes much smaller again: MocklessTester requires only 3.11k more tokens per iteration than PANTA. However, at the iteration level, MocklessTester requires approximately $51\,\%$ less time per iteration than PANTA.

These results show that MocklessTester incurs additional total cost mainly because it performs more iterations to pursue higher coverage, rather than because each iteration is inefficient. In absolute terms, the cost remains practical: MocklessTester requires less than two minutes per method on average and stays well below 60k tokens per method. Based on a conservative estimate using public API prices from providers such as OpenRouter and Alibaba, generating tests for the entire Defects4J benchmark across all 14 projects would cost only about \$12–\$45. Such overhead is acceptable for mockless test generation, especially considering the substantial effectiveness gains reported in RQ1. Moreover, the wall-clock time can be further reduced by using more powerful GPUs or commercial API-based LLM services.

Answer to RQ2: MocklessTester incurs a higher total token and time cost than PANTA because it performs more iterations to achieve higher coverage. However, its per-iteration token cost is competitive, and its per-iteration runtime is substantially lower than PANTA's. In absolute terms, MocklessTester remains practical, requiring less than two minutes and fewer than 60k tokens per method on average.

C. RQ3 Results (Ablation Study)

Table V reports the results of ablation studies, where we compare MocklessTester (denoted as M) with four ablated variants: *-UR* (removing usage retrieval), *-Sym* (removing symbol-level constraints), *-Prot* (removing protocol-level constraints), and *-Iter* (removing iteration-level constraints). By comparing MocklessTester across all four variants, we assess the individual contributions of usage retrieval, symbol-level constraints, protocol-level constraints, and iteration-level constraints to MocklessTester's effectiveness. Overall, we find that the full MocklessTester outperforms all variants in all metrics. The Wilcoxon signed-rank tests confirm the significance of all the gaps (*p-value*≤ 0.05) and the effect sizes are all large ($|\hat{A}_{12} - 0.5| > 0.21$), except two medium effect sizes on line coverage and branch coverage of *-Sym*.

Usage retrieval is the component that provides usage patterns for helping MocklessTester instantiate objects. When it is removed, average DepLC drops from 1197 to 903 on Defects4J and from 279 to 129 on Deps4J, the largest reduction of any ablation on both datasets. It also causes broad declines in line, branch, and mutation score: on Defects4J, average line coverage drops from 88.82 to 79.01, branch coverage from 83.74 to 72.25, and mutation score from 52.00 to 40.44; on Deps4J, the corresponding averages fall from 75.98 to 57.86, from 58.12 to 48.68, and from 47.09 to 44.23. This suggests that repository-specific usage examples do more than improve instantiation correctness: they

TABLE V

ABLATION STUDY: FULL MOCKLESSTESTER (*M*) VS FOUR ABLATIONS – *-UR* REMOVES USAGE PATTERN RETRIEVAL, *-Sym* REMOVES SYMBOL-LEVEL CONTRAINTS, *-Prot* REMOVES PROTOCOL-LEVEL CONTRAINTS, *-Iter* REMOVES ITERATION-LEVEL CONTRAINTS. EACH METRIC SPANS FIVE COLUMNS; BOLD MARKS THE VALUE THAT BEATS EVERY OTHER VARIANT WITHIN THAT PROJECT. PROJECT DEPLC IS THE COUNT OF DISTINCT LINES COVERED IN CLASSES OUTSIDE THE CUT. THE BOTTOM *Stat. test* ROW REPORTS THE TWO-SIDED PAIRED WILCOXON SIGNED-RANK *p-value* COMPARING *M* AGAINST EACH VARIANT ACROSS ALL PROJECTS AND THE VARGHA–DELANEY $\hat{A}_{12}$ EFFECT SIZE. "–" DENOTES NOT APPLICABLE.

Dataset	Project	Line %					Branch %					Mutation Score%					Project DepLC				
		M	-UR	-Sym	-Prot	-Iter	M	-UR	-Sym	-Prot	-Iter	M	-UR	-Sym	-Prot	-Iter	M	-UR	-Sym	-Prot	-Iter
Defects4J	Cli	96.18	89.75	**99.29**	97.12	97.36	89.88	77.16	91.93	**94.80**	93.97	57.23	34.10	**65.32**	55.49	61.27	**138**	89	84	107	84
	Codec	90.95	93.82	94.22	**94.92**	94.15	86.57	89.12	89.13	90.00	**90.13**	37.37	**49.38**	48.96	41.34	41.59	**415**	325	331	329	331
	Collections	**93.41**	74.92	76.50	71.84	73.32	**90.60**	72.97	73.14	68.04	68.27	**72.86**	51.90	54.57	46.98	48.66	**717**	470	518	473	506
	Compress	**85.89**	75.42	82.26	70.23	72.36	**78.77**	67.50	71.40	61.69	61.96	**40.76**	36.24	32.36	34.33	31.02	**2368**	1814	1707	1672	1559
	Csv	**96.52**	94.21	93.42	90.95	86.87	**87.90**	86.57	84.38	82.42	77.46	**29.36**	27.45	24.58	26.49	25.06	**84**	70	71	67	65
	Gson	86.86	85.35	**89.35**	75.60	87.52	81.96	83.94	**87.59**	72.15	79.44	**56.55**	53.31	55.25	31.13	52.92	587	542	569	**679**	646
	JCore	**80.49**	70.16	58.10	62.55	70.05	**74.69**	61.49	51.34	54.88	61.69	**34.42**	24.64	15.07	21.52	21.70	**1311**	770	563	876	908
	JDatabind	**81.64**	68.89	67.84	62.51	70.01	**76.10**	63.30	65.38	59.81	65.50	**60.22**	39.57	39.99	31.74	42.00	**3133**	2604	1905	1421	2735
	JXml	**80.23**	55.70	55.09	49.28	61.48	**80.12**	44.81	50.86	43.82	54.97	**47.09**	22.55	23.77	21.78	22.55	**221**	158	181	161	170
	Jsoup	89.76	90.59	93.47	**93.48**	92.10	80.42	77.56	82.08	**84.80**	81.48	58.84	54.65	**59.96**	57.20	58.43	1256	1331	1374	**1437**	1329
	JxPath	**84.45**	67.79	78.66	78.50	80.12	**80.27**	62.99	73.62	73.65	75.82	**58.48**	51.30	52.85	55.14	55.00	589	176	**1416**	1378	1280
	Lang	**86.39**	79.81	81.08	76.67	77.24	**83.00**	77.65	77.71	74.59	74.11	**49.06**	33.25	36.97	33.61	33.55	352	315	**361**	346	355
	Math	**92.72**	78.37	75.96	76.55	79.18	**89.10**	71.94	70.13	70.57	73.39	**48.95**	32.88	33.66	31.97	30.03	**3410**	2261	2316	2141	2347
	Time	**97.94**	81.36	84.47	88.52	89.60	**92.94**	74.54	79.58	84.34	84.95	**76.85**	54.90	60.22	65.95	64.17	**2172**	1714	1679	1761	1771
	Average	**88.82**	79.01	80.69	77.77	80.81	**83.74**	72.25	74.88	72.54	74.51	**52.00**	40.44	43.11	39.62	42.00	**1197**	903	934	918	1006
Deps4J	A2A	**87.04**	44.76	63.01	56.08	59.00	**59.35**	39.66	50.05	38.07	37.93	43.46	55.67	**65.88**	55.67	46.39	58	63	67	67	67
	ADK	76.25	79.36	**81.84**	75.28	78.57	61.31	65.11	**65.17**	62.80	62.59	48.32	50.16	**50.53**	45.37	48.69	101	26	**151**	26	47
	AgentScope	**76.52**	71.33	72.37	74.16	76.03	**69.02**	59.18	62.70	62.21	65.34	**54.45**	39.20	34.46	33.65	41.16	**717**	353	439	421	325
	FIT	79.76	80.24	**81.25**	80.93	80.51	70.62	71.45	72.33	**72.50**	71.27	**79.19**	76.11	71.88	72.18	79.08	**490**	205	231	218	231
	JQuick	**60.34**	13.60	25.39	20.00	35.61	**30.29**	7.99	18.86	14.89	29.47	10.05	0.00	0.00	0.00	**14.54**	**27**	0	0	0	13
	Average	**75.98**	57.86	64.77	61.29	65.94	**58.12**	48.68	53.82	50.09	53.32	**47.09**	44.23	44.55	41.37	45.97	**279**	129	178	146	137
Stat. test	*p-value*	–	<.001	0.006	<.001	0.005	–	<.001	0.003	<.001	0.002	–	0.002	0.023	<.001	0.008	–	<.001	0.020	0.020	0.020
	$\hat{A}_{12}$	–	0.789	0.684	0.789	0.684	–	0.789	0.684	0.737	0.737	–	0.842	0.737	0.895	0.737	–	0.895	0.737	0.789	0.737

also help the model generate tests that execute deeper behaviors once dependencies are reached.

Symbol-level constraints have a more moderate but still important effect. At the dataset level, *-Sym* lowers all four metrics on both benchmarks relative to the full method: on Defects4J, line coverage falls from 88.82% to 80.69%, branch coverage from 83.74% to 74.88%, mutation score from 52.00% to 43.11%, and DepLC from 1197 to 934; on Deps4J, the corresponding averages decrease from 75.98% to 64.77%, from 58.12% to 53.82%, from 47.09% to 44.55%, and from 279 to 178. Although the drop is less severe than for usage retrieval, it remains substantial. This is consistent with the role of symbol constraints in preventing invalid references and hallucinated APIs during repair. Interestingly, symbol-level constraints can occasionally hurt individual project metrics. For example, the full MocklessTester achieves a lower mutation score than *-Sym* on Cli in Defects4J and on A2A and ADK in Deps4J, and lower line or branch coverage on projects such as Cli, Codec, Gson, Jsoup, and ADK. Further investigation suggests that one plausible cause is ambiguous symbol resolution. For instance, ADK contains multiple classes with the same simple name across different packages and dependencies: `Schema` may refer to `com.google.genai.types.Schema`, `com.google.adk.tools.Annotations.Schema`, `com.google.cloud.bigquery.Schema`, or `org.apache.arrow.vector.types.pojo.Schema`. When the import context is incomplete, MocklessTester may deterministically prefer a project-local `com.google.adk.*` type over the intended external type due to package-prefix similarity. Thus, in projects with many homonymous APIs, symbolic resolution can occasionally select a wrong but plausible symbol. This limitation points to a future research direction: designing context-aware symbol resolution mechanisms that combine import history, call-site semantics, dependency usage patterns, and validation feedback to disambiguate symbols more reliably.

Protocol-level constraints have the largest impact among the ablated components. The *-Prot* variant achieves the lowest dataset-level line coverage and mutation score on both benchmarks: $77.77\,\%$ line coverage and $39.62\,\%$ mutation score on Defects4J, and $61.29\,\%$ line coverage and $41.37\,\%$ mutation score on Deps4J. The largest drops occur on protocol-heavy projects such as Collections, JDatabind, A2A, JQuick, and JXml, where a large portion of the CUTs involve stateful objects rather than simple method invocations. For example, JXml contains many stateful writer APIs whose deeper behavior is reachable only after satisfying specific protocol conditions. As discussed in Section I, a streaming writer must first establish the required name state by calling `setNextName(QName)` before invoking `writeStartObject()`. Removing protocol-level constraints therefore makes it harder for the LLMs to follow the valid call orders, causing line coverage on JXml to drop from $80.23\,\%$ to $49.28\,\%$, branch coverage from $80.12\,\%$ to $43.82\,\%$, and mutation score from $47.09\,\%$ to $21.78\,\%$. These results confirm that protocol-level constraints are especially important when the CUTs heavily depend on stateful objects.

Finally, *iteration-level constraints* appear to have the weakest influence among the four ablated components, yet their contribution remains substantial. The *-Iter* variant causes smaller drops than removing usage retrieval, symbol-level constraints, or protocol-level constraints, yet all metrics still decline substantially relative to the full method. On Defects4J, line coverage decreases from 88.82% to 80.81%, branch coverage from 83.74% to 74.51%, mutation score from 52.00% to 42.00%, and DepLC from 1197 to 1006. On Deps4J, the corresponding averages decrease from 75.98% to 65.94%, from 58.12% to 53.32%, from 47.09% to 45.97%, and from 279 to 137. Unlike usage retrieval, symbol-level constraints, and protocol-level constraints, which are derived from source-code analysis, iteration-level constraints are learned from previous generation and repair experience. They operate across iterations by preserving useful experience, discouraging repeated failures, and reusing successful repair patterns.

These results motivate a promising future direction: to incorporate richer memory mechanisms, such as GraphRAG [31] or dual-memory designs [32], to better organize past failures, successful repairs, dependency interactions, and reusable testing patterns.

Answer to RQ3: All four components improve the effectiveness of MocklessTester. Usage retrieval and protocol-level constraints have the largest impact, while symbol-level and iteration-level constraints provide additional gains by reducing hallucinated symbols and improving repair across iterations.

VI. Threats to Validity

Construct Validity. A potential threat to construct validity is whether the selected metrics accurately reflect the effectiveness and efficiency of a test generation approach. Standard metrics such as line coverage, branch coverage, and mutation score capture only part of test quality. For example, coverage may overestimate behavioral quality, while the mutation score can be influenced by the number and strength of assertions. To mitigate this threat, we use multiple complementary metrics, including line coverage, branch coverage, mutation score, project DepLC, number of iterations, token cost, and wall-clock time. Together, these metrics provide a more comprehensive view of both effectiveness and efficiency. In particular, we introduce the DepLC metric to measure the extent to which real dependency code is exercised beyond the CUT, which is especially relevant for evaluating mockless test generation.

Internal Validity. Threats to internal validity mainly arise from the configurations of MocklessTester and PANTA, as well as the choice of the backbone LLM. First, different hyperparameter settings may affect the comparison between the two tools. To mitigate this threat, we use identical settings for both MocklessTester and PANTA whenever possible, including the same context size, temperature, maximum output length, LLM, tokenizer, and hardware environment. We also set the same generation budget for both tools, based on the experimental setting recommended by PANTA, to ensure they have sufficient opportunity to converge across all projects and demonstrate their full capabilities. Second, the choice of LLM may influence the results. We select `Qwen3-Coder-30B-A3B-Instruct-FP8`, one of the latest and strongest small-scale open-source code models, considering both its coding capability and resource requirements. Using larger LLMs may further improve the absolute performance, but benchmarking different LLMs is not the objective of this study. Since both MocklessTester and PANTA use the same backbone model, stronger LLMs would likely benefit both approaches without altering the fairness of the comparison.

Conclusion Validity. Threats to conclusion validity arise from the stochastic nature of LLM-based test generation. Even with the same input, an LLM may generate different tests across runs, which can affect the observed effectiveness and efficiency. To mitigate this threat, we report average results across projects and apply the Wilcoxon signed-rank test to assess whether the differences between MocklessTester and PANTA are statistically significant. We also report effect sizes using the Vargha–Delaney $\hat{A}_{12}$, so that the results reflect not only statistical significance but also the magnitude of the observed differences. Although repeated executions could further reduce the influence of randomness, repeating every experiment multiple times would substantially increase the cost of an already expensive evaluation. As reported in RQ2, the per-iteration cost is substantial for both MocklessTester and PANTA: on Defects4J, each iteration consumes 61.42k tokens and 254.60 seconds for MocklessTester, compared with 50.17k tokens and 587.65 seconds for PANTA; on Deps4J, each iteration consumes 53.25k tokens and 147.22 seconds for MocklessTester, compared with 50.14k tokens and 222.58 seconds for PANTA. Each additional repetition would multiply the number of LLM calls while also repeating compilation, execution, coverage collection, mutation analysis, and repair. Given that our evaluation already covers many CUTs across diverse projects and shows consistent trends across benchmarks, we prioritize breadth across CUTs and projects over repeated executions of the same targets.

External Validity. Threats to external validity concern whether our results generalize to other Java projects. To mitigate this threat, we evaluate MocklessTester on Defects4J, a widely used Java testing benchmark containing 14 projects from diverse domains. However, because Defects4J was curated more than a decade ago, some of its code may have appeared in the training corpus of Qwen3-Coder, raising a potential data leakage concern. To further assess generalizability and reduce this risk, we curate Deps4J, a new benchmark comprising Java projects created after the Qwen3-Coder training cutoff date. The consistent results of MocklessTester on both Defects4J and Deps4J suggest that the observed improvements are not limited to a single benchmark or caused by benchmark memorization. Nevertheless, our evaluation remains limited to Java projects and a single backbone LLM, so further studies across other languages, repositories, and models are needed in the future.

VII. Related Work

Software testing is a fundamental engineering task for ensuring software quality and mitigating release risks [8]. Manually creating tests is labor-intensive and error-prone, motivating the development of various automated test generation (ATG) approaches. Early ATG approaches predominantly rely on search-based algorithms [4, 33, 34, 5] and symbolic/concolic execution [35, 36]. These approaches produce tests exhibiting high coverage, while their understanding of the code (CUT) is often limited [8]. Consequently, these approaches struggle to generate semantically meaningful tests involving dependencies, stateful objects, and domain-specific behaviors.

Recently, LLMs have garnered substantial attention for their success across a wide range of software engineering tasks, including code generation, vulnerability detection, and test generation. Pretrained on large, diverse corpora, LLMs inherently possess rich knowledge of programming languages, APIs, and software development practices, making them promising candidates for generating tests for complex projects. A growing body of work has explored LLM-based test generation [8]. Existing approaches show that LLMs can generate complex input with domain-specific semantics [37, 38], construct effective test prefixes [39, 40, 7, 2], and produce reasonable mock implementations for external dependencies [41, 42].

Among LLM-based approaches, CANDOR [2] and PANTA [7] are the most closely related to MocklessTester. CANDOR is the first generic framework using multi-agent LLMs to generate test cases for Java programs. It decomposes the complex test generation task into specialized agents, such as `planner`, `tester`, and `fixer`, thereby laying the foundation for subsequent multi-agent test generation approaches, including MocklessTester. However,

CANDOR primarily aims to demonstrate the feasibility of end-to-end Java test generation with multi-agent LLMs. It targets method-level programs without external dependencies, such as HumanEvalJava, and therefore does not address the challenges of mockless test generation.

PANTA advances LLM-based Java test generation from method-level to complex scenarios involving dependencies. It leverages path information in prompts to guide LLMs more precisely by indicating which execution paths remain uncovered. It also ranks paths to encourage LLMs to prioritize important ones, such as those spanning more lines of code. PANTA achieves substantial improvements over the baseline Symprompt [24] and reaches SOTA performance on the Defects4J benchmark. Despite its success on complex benchmarks such as Defects4J, PANTA does not explicitly address external dependencies. In practice, many such dependencies are handled by invoking mocking frameworks such as Mockito to simulate dependent components, rather than exercising the real dependency code.

Our method, MocklessTester, follows this research direction and aims to generate mockless Java unit tests. Similar to CANDOR and PANTA, MocklessTester adopts a multi-agent design and incorporates path information into prompts to guide LLMs in generation. Unlike prior approaches that rely on mocking frameworks to handle external dependencies, MocklessTester directly exercises real code, enabling tests to cover more realistic execution behavior. However, handling real dependencies is challenging because LLMs may hallucinate APIs, object states, method usages, or dependency interactions. To mitigate such hallucinations, MocklessTester synergistically combines multiple strategies to provide sufficient contextual information and enforce constraints on LLM-generated tests. To the best of our knowledge, MocklessTester is the first mockless testing framework that explicitly targets external dependencies in mockless Java unit test generation.

VIII. Conclusion

This paper presents MocklessTester, a mockless unit test generation approach for Java. Unlike prior LLM-based approaches that rely on mocking frameworks, MocklessTester directly exercises real project dependencies by combining context-enriched generation with constraint-enforced repair. It addresses two major sources of hallucination in mockless test generation: *not knowing*, mitigated by dependency-usage retrieval, and *not following*, mitigated by symbol-, protocol-, and iteration-level constraints.

Our evaluation on Defects4J and Deps4J shows that MocklessTester consistently improves test quality and dependency coverage over the SOTA baseline PANTA. On Defects4J, MocklessTester improves average line coverage from 68.83% to 88.82%, branch coverage from 58.84% to 83.74%, mutation score from 38.33% to 52.00%, and project DepLC from 819 to 1197, while generating fewer tests. On Deps4J, MocklessTester again improves line, branch, and dependency-reaching coverage, while maintaining comparable mutation scores. Although these gains require a higher total generation cost in terms of token and time, the cost remains practical, averaging 108.97 seconds and 26.59k tokens per method on Defects4J, and 69.85 seconds and 25.46k tokens per method on Deps4J. An ablation study shows that all four major components of MocklessTester contribute positively, including usage retrieval, symbol-level constraints, protocol-level constraints, and iteration-level constraints.

Future directions include developing richer memory mechanisms to better organize past failures and successful repairs, improving context-aware symbol resolution for projects with many homonymous APIs, evaluating the readability and maintainability of generated tests, and extending MocklessTester to other strongly typed languages.

Acknowledgement

This work has emanated from research jointly funded by Taighde Éireann – Research Ireland under Grant number 13/RC/2094_2 and by Huawei Technologies Co., Ltd. Lionel Briand is also supported by the Natural Sciences and Engineering Research Council of Canada. For the purpose of Open Access, the authors have applied a CC BY public copyright licence to any Author Accepted Manuscript version arising from this submission.

References

[1] K. Beck, *Test driven development: By example*. Addison-Wesley Professional, 2022.

[2] Q. Xu, G. Wang, L. Briand, and K. Liu, "Hallucination to consensus: Multi-agent llms for end-to-end junit test generation," *ACM Transactions on Software Engineering and Methodology*, 2026.

[3] B. Yu, Y. Cao, Y. Zhang, L. Lin, J. Xu, Z. Zhong, Q. Xu, G. Wang, J. Cao, S.-C. Cheung *et al.*, "Swe-abs: Adversarial benchmark strengthening exposes inflated success rates on test-based benchmark," *arXiv preprint arXiv:2603.00520*, 2026.

[4] G. Fraser and A. Arcuri, "A large-scale evaluation of automated unit test generation using evosuite," *ACM Trans. Softw. Eng. Methodol.*, vol. 24, no. 2, Dec. 2014. [Online]. Available: https://doi.org/10.1145/2685612

[5] C. Pacheco and M. D. Ernst, "Randoop: feedback-directed random testing for java," in *Companion to the 22nd ACM SIGPLAN conference on Object-oriented programming systems and applications companion*, 2007, pp. 815–816.

[6] G. Wang, Q. Xu, L. C. Briand, and K. Liu, "Mutation-guided unit test generation with a large language model," *arXiv preprint arXiv:2506.02954*, 2025.

[7] S. Gu, N. Nashid, and A. Mesbah, "Llm test generation via iterative hybrid program analysis," *arXiv preprint arXiv:2503.13580*, 2025.

[8] B. Chu, Y. Feng, K. Liu, Z. Guo, Y. Zhang, H. Shi, Z. Nan, and B. Xu, "Large language models for unit test generation: Achievements, challenges, and opportunities," *arXiv preprint arXiv:2511.21382*, 2025.

[9] J. Altmayer Pizzorno and E. D. Berger, "Coverup: Effective high coverage test generation for python," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 2897–2919, 2025.

[10] S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty, and S. K. Lahiri, "Llm-based test-driven interactive code generation: User study and empirical evaluation," *IEEE Transactions on Software Engineering*, vol. 50, no. 9, pp. 2254–2268, 2024.

[11] W. Wang, C. Yang, Z. Wang, Y. Huang, Z. Chu, D. Song, L. Zhang, A. R. Chen, and L. Ma, "TestEval: Benchmarking large language models for test case generation," in *Findings of the Association for Computational Linguistics: NAACL*

2025, L. Chiruzzo, A. Ritter, and L. Wang, Eds. Albuquerque, New Mexico: Association for Computational Linguistics, Apr. 2025, pp. 3547–3562. [Online]. Available: https://aclanthology.org/2025.findings-naacl.197/

[12] Z. Nan, Z. Guo, K. Liu, and X. Xia, "Test intention guided llm-based unit test generation," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering*, ser. ICSE '25. IEEE Press, 2025, p. 1026–1038. [Online]. Available: https://doi.org/10.1109/ICSE55347.2025.00243

[13] D. Spadini, M. Aniche, M. Bruntink, and A. Bacchelli, "To mock or not to mock? an empirical study on mocking practices," in *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*, 2017, pp. 402–412.

[14] S. B. Hossain and M. Dwyer, "Togll: Correct and strong test oracle generation with llms," *arXiv preprint arXiv:2405.03786*, 2024.

[15] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," in *Proceedings of the 2014 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2014, pp. 590–604.

[16] D. Das, N. S. Mathews, A. Mathai, S. Tamilselvam, K. Sedamaki, S. Chimalakonda, and A. Kumar, "Comex: A tool for generating customized source code representations," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 2054–2057.

[17] Google, "Agent development kit for java (adk-java)," https://github.com/google/adk-java, 2025, accessed March 2026.

[18] Huawei ModelEngine Group, "Fit framework: Model-engineering utilities for java," https://github.com/ModelEngine-Group/fit-framework, 2025, accessed March 2026.

[19] AgentScope Contributors, "Agentscope: A flexible yet robust multi-agent platform (java runtime)," https://github.com/agentscope-ai/agentscope-java, 2025, accessed March 2026.

[20] A2A Project Contributors, "Agent-to-agent (a2a) sdk for java," https://github.com/a2aproject/a2a-java, 2025, accessed March 2026.

[21] H. Pao, "jquick-curl: An antlr-based curl command parser for java," https://github.com/paohaijiao/jquick-curl, 2025, accessed March 2026.

[22] Y. Chen, Z. Hu, C. Zhi, J. Han, S. Deng, and J. Yin, "Chatunitest: A framework for llm-based test generation," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024, pp. 572–576.

[23] A. Deljouyi, R. Koohestani, M. Izadi, and A. Zaidman, "Leveraging large language models for enhancing the understandability of generated unit tests," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering*, ser. ICSE '25. IEEE Press, 2025, p. 1449–1461. [Online]. Available: https://doi.org/10.1109/ICSE55347.2025.00032

[24] G. Ryan, S. Jain, M. Shang, S. Wang, X. Ma, M. K. Ramanathan, and B. Ray, "Code-aware prompting: A study of coverage-guided test generation in regression setting using llm," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 951–971, 2024.

[25] H. Coles, T. Laurent, C. Henard, M. Papadakis, and A. Ventresque, "PIT: A practical mutation testing tool for Java (demo)," in *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2016, pp. 449–452.

[26] A. Arcuri and L. Briand, "A practical guide for using statistical tests to assess randomized algorithms in software engineering," in *Proceedings of the 33rd international conference on software engineering*, 2011, pp. 1–10.

[27] A. Vargha and H. D. Delaney, "A critique and improvement of the cl common language effect size statistics of mcgraw and wong," *Journal of Educational and Behavioral Statistics*, vol. 25, no. 2, pp. 101–132, 2000.

[28] "Langchain official website," Accessed: 2025. [Online]. Available: https://www.langchain.com/

[29] Qwen Team, "Qwen3-Coder: Agentic coding models," https://qwenlm.github.io/blog/qwen3-coder/, 2025, accessed March 2026.

[30] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th symposium on operating systems principles*, 2023, pp. 611–626.

[31] H. Qian, Z. Liu, P. Zhang, K. Mao, D. Lian, Z. Dou, and T. Huang, "Memorag: Boosting long context processing with global memory-enhanced retrieval augmentation," in *Proceedings of the ACM on Web Conference 2025*, 2025, pp. 2366–2377.

[32] F. Mu, J. Wang, L. Shi, S. Wang, S. Li, and Q. Wang, "Experepair: Dual-memory enhanced llm-based repository-level program repair," *arXiv preprint arXiv:2506.10484*, 2025.

[33] R. S. Herlim, Y. Kim, and M. Kim, "Citrus: Automated unit testing tool for real-world c++ programs," in *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE Computer Society, 2022, pp. 400–410.

[34] S. Lukasczyk and G. Fraser, "Pynguin: Automated unit test generation for python," in *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, 2022, pp. 168–172.

[35] T. Chen, X.-S. Zhang, X.-L. Ji, C. Zhu, Y. Bai, and Y. Wu, "Test generation for embedded executables via concolic execution in a real environment," *IEEE Transactions on Reliability*, vol. 64, no. 1, pp. 284–296, 2014.

[36] P. Garg, F. Ivančić, G. Balakrishnan, N. Maeda, and A. Gupta, "Feedback-directed unit test generation for c/c++ using concolic execution," in *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 2013, pp. 132–141.

[37] A. Guzu, G. Nicolae, H. Cucu, and C. Burileanu, "Large language models for c test case generation: A comparative analysis," *Electronics*, vol. 14, no. 11, p. 2284, 2025.

[38] N. Huynh and B. Lin, "Large language models for code generation: A comprehensive survey of challenges, techniques, evaluation, and applications," *arXiv preprint arXiv:2503.01245*, 2025.

[39] R. Pan, M. Kim, R. Krishna, R. Pavuluri, and S. Sinha, "Aster: Natural and multi-language unit test generation with llms," in *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice*

(ICSE-SEIP). IEEE, 2025, pp. 413–424.

[40] Y. Zhang, Q. Lu, K. Liu, W. Dou, J. Zhu, L. Qian, C. Zhang, Z. Lin, and J. Wei, "Citywalk: Enhancing llm-based c++ unit test generation via project-dependency awareness and language-specific knowledge," *ACM Transactions on Software Engineering and Methodology*, 2025.

[41] D. Gorla, S. Kumar, P. N. R. Lorenzini, and A. Alipourfaz, "Cubetesterai: Automated junit test generation using the llama model," in *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2025, pp. 565–576.

[42] S. Roy Chowdhury, G. Sridhara, A. Raghavan, J. Bose, S. Mazumdar, H. Singh, S. B. Sugumaran, and R. Britto, "Static program analysis guided llm based unit test generation," in *Proceedings of the 8th International Conference on Data Science and Management of Data (12th ACM IKDD CODS and 30th COMAD)*, 2024, pp. 279–283.